

Parking Lot - How it works

Home notes + DESIGN.md + source code | compact print

In one line - Car comes in -> find a free spot -> give ticket -> on exit charge money -> free the spot.

Story - how this LLD works

Imagine a mall parking. You drive to the entry gate. The system looks for a free spot that matches your vehicle (bike spot for bike, car spot for car). If it finds one, it locks that spot and prints a ticket. When you leave, it looks at how long you stayed, calculates the fee, takes payment, then unlocks the spot for the next person.

Who does what

ParkingLot: The brain. Remembers all floors, spots, and active tickets. Only ONE exists (Singleton).

EntryGate / ExitGate: The doors. They don't do heavy work - they just call ParkingLot.

ParkingFloor: One floor of the building. Holds many ParkingSpots.

ParkingSpot: One parking place. Has a lock so two cars cannot grab it at the same time.

Vehicle (Car/Bike/Truck): What is parking. Type decides which spot is allowed.

Ticket: Your receipt for this visit: which floor, which spot, entry time.

PricingStrategy: How to calculate money (peak hours vs flat event rate). Easy to swap.

PaymentStrategy: How you pay (cash / card / UPI). Easy to swap.

Main pieces

ParkingLot (brain)

- floors
- activeTickets
- pricingStrategy
- park / unpark

ParkingFloor

- spots list
- find free spot

ParkingSpot

- type allowed
- occupied?
- tryOccupy / vacate

Ticket

- id, time
- vehicle
- floor + spot

Step-by-step at runtime

1. You arrive with a Bike at EntryGate.
2. EntryGate asks ParkingLot: park this bike.
3. ParkingLot checks Floor 1 spots. Finds a BIKE spot that is free.
4. Spot.tryOccupy() locks it (thread-safe). Nobody else can take it.
5. A Ticket is created (ticket id + entry time + spot id) and saved in activeTickets.
6. Later at ExitGate you give ticket id + choose UPI.
7. ParkingLot asks PricingStrategy: how much for this bike from entry to now?
8. PaymentStrategy (UPI) processes payment. If OK, spot is vacated and ticket removed.
9. If payment fails, spot stays occupied - you did not leave yet.

Why this design (plain words)

Why Singleton? - There is one physical parking lot. All gates must share the same spots and tickets.

Why Strategy for price/pay? - Tomorrow mall may change rates or add Wallet pay. Add a new class - don't rewrite ParkingLot.

Why Factory? - Pass VehicleType.CAR instead of writing if/else everywhere to create Car/Bike/Truck.

Why lock on Spot? - Two threads (two bikes) might race for the last spot. Lock makes only one win.

Remember

- Spot type must match vehicle type (car cannot use truck spot in this design).
- Ticket stores floor_id + spot_id so exit knows what to free.
- Payment failure = no unpark.
- Demo main.py parks two bikes concurrently to show the lock.

OOP in this project

Encapsulation - Spot hides occupied flag. Outside world only calls tryOccupy/vacate.

Abstraction - PricingStrategy only says calculateFee - not how peak hours work.

Inheritance - Car, Bike, Truck are kinds of Vehicle.

Polymorphism - ParkingLot calls calculateFee without caring if it is TimeBased or EventBased.

DESIGN.md - full file

Complete markdown from lld/parking_lot/DESIGN.md (Q&A for learning).

Full DESIGN.md (same file as in this folder) - DESIGN.md

Designing a Parking Lot - LLD Q&A

Run: `python -m lld.parking_lot.main`

Patterns: Singleton · Strategy · Factory · Thread safety

Q1. What problem are we solving?

A. Design a multi-floor parking lot that:

Accepts vehicles (Bike / Car / Truck) at an entry gate

Finds a matching free spot and issues a ticket

On exit, calculates a fee using a pricing strategy

Accepts payment (Cash / UPI / Card)

Vacates the spot only after successful payment

Also handle concurrent park attempts safely so two vehicles cannot claim the same spot.

Q2. What are the core entities?

- Entity - Responsibility
- Vehicle (Car, Bike, Truck) - Vehicle number + type
- ParkingSpot - One spot for one vehicle type; thread-safe occupy/vacate
- ParkingFloor - Collection of spots; finds first free matching spot
- Ticket - Session: ticket id, entry time, vehicle, floor, spot, payment status
- EntryGate / ExitGate - Thin facades over ParkingLot
- ParkingLot - Singleton orchestrator - floors, active tickets, pricing

Q3. How is the package structured?

```
parking_lot/
|-- model/      # Domain entities (Vehicle, Spot, Floor, Ticket, Gate)
|-- service/    # ParkingLot, PaymentProcessor
|-- strategy/   # Pricing + Payment algorithms
|-- factory/    # Vehicle / Pricing / Payment creation
|-- enums/     # VehicleType, PaymentMode, PricingStrategyType, ...
|-- utils/     # Date-time parsing
```

- Layer - Why it exists
- model - Pure domain objects - little business logic
- service - Orchestration and state
- strategy - Swappable fee & payment algorithms (OCP)
- factory - Enum -> concrete class without if/else in callers
- enums - Type-safe constants

Q4. Why Singleton for `ParkingLot`?

A. There is one physical lot. All entry/exit gates must share the same floors, spots, and active tickets.

```
ParkingLot.get_instance() # shared by EntryGate and ExitGate
```

Gates call `ParkingLot.get_instance()` instead of holding separate lot instances.

Q5. Why Strategy for pricing and payment?

A. Fee rules and payment channels change independently. Strategy lets us add new algorithms without changing `ParkingLot`.

Pricing: `TimeBasedPricing` (peak/off-peak hourly) vs `EventBasedPricing` (flat hourly)

Payment: `CashPayment`, `CardPayment`, `UpiPayment`

`ParkingLot` only depends on the interface:

```
fee = self.pricing_strategy.calculate_fee(vehicle_type, entry_time, exit_time)
```

Q6. Why Factory?

A. Callers pass an enum (`VehicleType.CAR`, `PaymentMode.UPI`) instead of constructing concrete classes. Creation logic stays in one place.

| Factory | Maps |

|-----|-----|

| `VehicleFactory` | `VehicleType` -> Car / Bike / Truck |

| `PricingStrategyFactory` | `PricingStrategyType` -> Time / Event based |

| `PaymentStrategyFactory` | `PaymentMode` -> Cash / UPI / Card |

Q7. What is the entry (park) flow?

```
EntryGate.park_vehicle(vehicle, entry_time)
-> ParkingLot.park_vehicle(...)
  -> for each floor: find_available_spot(vehicle.type)
    -> ParkingSpot.try_occupy() # locked
  -> create Ticket (UUID), store in active_tickets
  -> return Ticket (or None if full)
```

Spot rule: Exact type match only (Car cannot use Truck spot).

Allocation: First-fit across floors.

Q8. What is the exit (unpark) flow?

```
ExitGate.unpark_vehicle(ticket_id, exit_time, payment_mode)
-> lookup ticket
-> pricing_strategy.calculate_fee(...)
-> PaymentStrategyFactory.get(mode) -> PaymentProcessor.pay(...)
-> on success: vacate spot, remove ticket
-> on failure: abort - spot stays occupied
```

Q9. How is concurrency handled?

A. Each ParkingSpot has a threading.Lock. try_occupy() atomically claims the spot:

```
def try_occupy(self) -> bool:
    with self._lock:
        if not self._occupied:
            self._occupied = True
            return True
        return False
```

main.py parks two bikes on two threads to demonstrate this. With one bike spot, only one succeeds.

Q10. TIME_BASED vs EVENT_BASED pricing?

- Time-based - Event-based
- Logic - Hour-by-hour peak (8 AM-5 PM) vs non-peak | Flat rate per hour
- Bike - Peak Rs.15 / Non-peak Rs.10 | Rs.30/hr
- Car - Peak Rs.30 / Non-peak Rs.20 | Rs.50/hr
- Truck - Peak Rs.50 / Non-peak Rs.30 | Rs.70/hr
- Default in ParkingLot is TIME_BASED; the demo switches to EVENT_BASED.

Q11. How do you extend this design?

- Add... - Steps
- New vehicle (e.g. EV) - Enum + model class + factory branch + rates + spots
- New pricing - Implement PricingStrategy + factory + enum
- New payment - Implement PaymentStrategy + factory + enum
- ParkingLot.unpark_vehicle does not need changes - it already depends on interfaces.

Q12. Common interview follow-ups

Q. What if payment fails?

Spot stays occupied; ticket remains in active_tickets.

Q. Where is ticket linked to spot?

Ticket stores floor_id + spot_id; exit looks them up to vacate.

Q. What's not modeled?

Reservations, handicap/EV charging, size hierarchy (Car in Truck spot), persistent storage, receipt entity.

Q. Is the Singleton thread-safe?

Lazy init without a lock - fine for demo; production would use a lock or module-level instance.

Source code - lld/parking_lot/

29 Python files below (compact font to save pages). Read with the notes: find the class name from the cast, then open that file here.

- Tip: start at main.py, then service/, then model/ + strategy/.

FILE: main.py (55 lines)

```
1| import threading
2| from datetime import datetime
3|
4| from .enums.pricing_strategy_type import PricingStrategyType
5| from .enums.vehicle_type import VehicleType
6| from .factory.pricing_strategy_factory import PricingStrategyFactory
7| from .factory.vehicle_factory import VehicleFactory
8| from .model.entry_gate import EntryGate
9| from .model.exit_gate import ExitGate
10| from .model.parking_floor import ParkingFloor
11| from .model.parking_spot import ParkingSpot
12| from .service.parking_lot import ParkingLot
13| from .utils.date_time_parser import DateTimeParser
14|
15|
16| def main() -> None:
17|     lot = ParkingLot.get_instance()
18|     entry_gate = EntryGate("EG1")
19|     exit_gate = ExitGate("XG1")
20|
21|     lot.pricing_strategy = PricingStrategyFactory.get(
22|         PricingStrategyType["EVENT_BASED"]
23|     )
24|
25|     floor1 = ParkingFloor("Floor1")
26|     floor1.add_spot(ParkingSpot("F1S1", VehicleType.BIKE))
27|     floor1.add_spot(ParkingSpot("F1S2", VehicleType.CAR))
28|     floor1.add_spot(ParkingSpot("F1S3", VehicleType.TRUCK))
29|     floor1.add_spot(ParkingSpot("F1S4", VehicleType.CAR))
30|     lot.add_floor(floor1)
31|
32|     print("-----")
33|
34|     bike1 = VehicleFactory.create("KA01AB1234", VehicleType.BIKE)
35|     bike2 = VehicleFactory.create("KA01AB5678", VehicleType.BIKE)
36|     entry_time = DateTimeParser.parse("21 May 7:30 AM 2025")
37|     print(entry_time.replace(minute=0, second=0, microsecond=0))
38|
39|     t1 = threading.Thread(
40|         target=entry_gate.park_vehicle,
41|         args=(bike1, entry_time),
42|     )
43|     t2 = threading.Thread(
44|         target=entry_gate.park_vehicle,
45|         args=(bike2, entry_time),
46|     )
47|
48|     t1.start()
49|     t2.start()
50|     t1.join()
51|     t2.join()
52|
53|
54| if __name__ == "__main__":
55|     main()
```

FILE: service/parking_lot.py (94 lines)

```
1| import uuid
2| from datetime import datetime
3|
4| from ..enums.payment_mode import PaymentMode
5| from ..enums.pricing_strategy_type import PricingStrategyType
6| from ..factory.payment_strategy_factory import PaymentStrategyFactory
7| from ..factory.pricing_strategy_factory import PricingStrategyFactory
8| from ..model.parking_floor import ParkingFloor
9| from ..model.parking_spot import ParkingSpot
10| from ..model.ticket import Ticket
11| from ..model.vehicle import Vehicle
12| from ..strategy.pricing.pricing_strategy import PricingStrategy
13| from .payment_processor import PaymentProcessor
14|
15|
16| class ParkingLot:
17|     _instance: "ParkingLot | None" = None
18|
19|     def __init__(self) -> None:
20|         self.floors: dict[str, ParkingFloor] = {}
21|         self.active_tickets: dict[str, Ticket] = {}
22|         self.pricing_strategy: PricingStrategy = PricingStrategyFactory.get(
23|             PricingStrategyType.TIME_BASED
24|         )
25|
26|     @classmethod
27|     def get_instance(cls) -> "ParkingLot":
28|         if cls._instance is None:
29|             cls._instance = ParkingLot()
30|         return cls._instance
31|
32|     def add_floor(self, floor: ParkingFloor) -> None:
33|         self.floors[floor.id] = floor
34|
35|     def park_vehicle(self, vehicle: Vehicle, entry_time: datetime) -> Ticket | None:
```

```

36|     for floor in self.floors.values():
37|         spot = floor.find_available_spot(vehicle.type)
38|
39|         if spot is not None:
40|             ticket_id = str(uuid.uuid4())
41|             ticket = Ticket(
42|                 ticket_id=ticket_id,
43|                 entry_time=entry_time,
44|                 vehicle=vehicle,
45|                 floor_id=floor.id,
46|                 spot_id=spot.id,
47|             )
48|
49|             self.active_tickets[ticket_id] = ticket
50|             print(f"Vehicle parked. Ticket: {ticket_id}")
51|             return ticket
52|
53|     print(f"No spot available for vehicle type: {vehicle.type}")
54|     return None
55|
56| def unpark_vehicle(
57|     self,
58|     ticket_id: str,
59|     exit_time: datetime,
60|     payment_mode: PaymentMode,
61| ) -> None:
62|     ticket = self.active_tickets.get(ticket_id)
63|     if ticket is None:
64|         print("Invalid ticket ID.")
65|         return
66|
67|     fee = self.pricing_strategy.calculate_fee(
68|         ticket.vehicle.type,
69|         ticket.entry_time,
70|         exit_time,
71|     )
72|
73|     strategy = PaymentStrategyFactory.get(payment_mode)
74|     processor = PaymentProcessor(strategy)
75|     paid = processor.pay(ticket, fee)
76|
77|     if not paid:
78|         print("Vehicle cannot exit. Payment unsuccessful.")
79|         return
80|
81|     spot = self.floors[ticket.floor_id].spots[ticket.spot_id]
82|     spot.vacate()
83|     del self.active_tickets[ticket_id]
84|
85|     print(f"Vehicle exited. Fee charged: Rs.{fee}")
86|
87| def print_status(self) -> None:
88|     for floor_id, floor in self.floors.items():
89|         print(f"Floor: {floor_id}")
90|         for spot in floor.spots.values():
91|             status = "Occupied" if spot.is_occupied() else "Free"
92|             print(
93|                 f"Spot {spot.id} [{spot.allowed_type}] - {status}"
94|             )

```

FILE: service/payment_processor.py (17 lines)

```

1| from ..enums.payment_status import PaymentStatus
2| from ..model.ticket import Ticket
3| from ..strategy.payment.payment_strategy import PaymentStrategy
4|
5|
6| class PaymentProcessor:
7|     def __init__(self, strategy: PaymentStrategy) -> None:
8|         self._strategy = strategy
9|
10|     def pay(self, ticket: Ticket, amount: float) -> bool:
11|         success = self._strategy.process_payment(ticket, amount)
12|         if success:
13|             ticket.payment_status = PaymentStatus.SUCCESS
14|         else:
15|             ticket.payment_status = PaymentStatus.FAILED
16|             print(f"Payment failed for ticket: {ticket.ticket_id}")
17|         return success

```

FILE: model/bike.py (7 lines)

```

1| from ..enums.vehicle_type import VehicleType
2| from .vehicle import Vehicle
3|
4|
5| class Bike(Vehicle):
6|     def __init__(self, number: str) -> None:
7|         super().__init__(number=number, type=VehicleType.BIKE)

```

FILE: model/car.py (7 lines)

```

1| from ..enums.vehicle_type import VehicleType
2| from .vehicle import Vehicle
3|
4|
5| class Car(Vehicle):
6|     def __init__(self, number: str) -> None:
7|         super().__init__(number=number, type=VehicleType.CAR)

```

FILE: model/entry_gate.py (18 lines)

```

1| from datetime import datetime
2|
3| from ..enums.gate_type import GateType
4| from ..service.parking_lot import ParkingLot
5| from .gate import Gate
6| from .ticket import Ticket
7| from .vehicle import Vehicle
8|
9|
10| class EntryGate(Gate):
11|     def __init__(self, gate_id: str) -> None:
12|         super().__init__(id=gate_id)
13|
14|     def get_type(self) -> GateType:
15|         return GateType.ENTRY
16|
17|     def park_vehicle(self, vehicle: Vehicle, entry_time: datetime) -> Ticket | None:
18|         return ParkingLot.get_instance().park_vehicle(vehicle, entry_time)

```

FILE: model/exit_gate.py (22 lines)

```

1| from datetime import datetime
2|
3| from ..enums.gate_type import GateType
4| from ..enums.payment_mode import PaymentMode
5| from ..service.parking_lot import ParkingLot
6| from .gate import Gate
7|
8|
9| class ExitGate(Gate):
10|     def __init__(self, gate_id: str) -> None:
11|         super().__init__(id=gate_id)
12|
13|     def get_type(self) -> GateType:
14|         return GateType.EXIT
15|
16|     def unpark_vehicle(
17|         self,
18|         ticket_id: str,
19|         exit_time: datetime,
20|         payment_mode: PaymentMode,
21|     ) -> None:
22|         ParkingLot.get_instance().unpark_vehicle(ticket_id, exit_time, payment_mode)

```

FILE: model/gate.py (13 lines)

```

1| from abc import ABC, abstractmethod
2| from dataclasses import dataclass
3|
4| from ..enums.gate_type import GateType
5|
6|
7| @dataclass
8| class Gate(ABC):
9|     id: str
10|
11|     @abstractmethod
12|     def get_type(self) -> GateType:
13|         pass

```

FILE: model/parking_floor.py (17 lines)

```

1| from ..enums.vehicle_type import VehicleType
2| from .parking_spot import ParkingSpot
3|
4|
5| class ParkingFloor:
6|     def __init__(self, floor_id: str) -> None:
7|         self.id = floor_id
8|         self.spots: dict[str, ParkingSpot] = {}
9|
10|     def add_spot(self, spot: ParkingSpot) -> None:
11|         self.spots[spot.id] = spot
12|
13|     def find_available_spot(self, vehicle_type: VehicleType) -> ParkingSpot | None:
14|         for spot in self.spots.values():
15|             if spot.allowed_type == vehicle_type and spot.try_occupy():
16|                 return spot
17|         return None

```

FILE: model/parking_spot.py (26 lines)

```

1| import threading
2|
3| from ..enums.vehicle_type import VehicleType
4|
5|
6| class ParkingSpot:
7|     def __init__(self, spot_id: str, allowed_type: VehicleType) -> None:
8|         self.id = spot_id
9|         self.allowed_type = allowed_type
10|         self.occupied = False
11|         self._lock = threading.Lock()
12|
13|     def try_occupy(self) -> bool:
14|         with self._lock:
15|             if not self.occupied:
16|                 self.occupied = True
17|                 return True
18|             return False
19|

```

```
20| def vacate(self) -> None:
21|     with self._lock:
22|         self._occupied = False
23|
24| def is_occupied(self) -> bool:
25|     with self._lock:
26|         return self._occupied
```

FILE: model/ticket.py (15 lines)

```
1| from dataclasses import dataclass, field
2| from datetime import datetime
3|
4| from ..enums.payment_status import PaymentStatus
5| from .vehicle import Vehicle
6|
7|
8| @dataclass
9| class Ticket:
10|     ticket_id: str
11|     entry_time: datetime
12|     vehicle: Vehicle
13|     floor_id: str
14|     spot_id: str
15|     payment_status: PaymentStatus = field(default=PaymentStatus.PENDING)
```

FILE: model/truck.py (7 lines)

```
1| from ..enums.vehicle_type import VehicleType
2| from .vehicle import Vehicle
3|
4|
5| class Truck(Vehicle):
6|     def __init__(self, number: str) -> None:
7|         super().__init__(number=number, type=VehicleType.TRUCK)
```

FILE: model/vehicle.py (10 lines)

```
1| from abc import ABC
2| from dataclasses import dataclass
3|
4| from ..enums.vehicle_type import VehicleType
5|
6|
7| @dataclass
8| class Vehicle(ABC):
9|     number: str
10|     type: VehicleType
```

FILE: strategy/payment/card_payment.py (8 lines)

```
1| from ...model.ticket import Ticket
2| from .payment_strategy import PaymentStrategy
3|
4|
5| class CardPayment(PaymentStrategy):
6|     def process_payment(self, ticket: Ticket, amount: float) -> bool:
7|         print(f"Paid Rs.{amount} for ticket {ticket.ticket_id} via Card.")
8|         return True
```

FILE: strategy/payment/cash_payment.py (8 lines)

```
1| from ...model.ticket import Ticket
2| from .payment_strategy import PaymentStrategy
3|
4|
5| class CashPayment(PaymentStrategy):
6|     def process_payment(self, ticket: Ticket, amount: float) -> bool:
7|         print(f"Paid Rs.{amount} for ticket {ticket.ticket_id} via Cash.")
8|         return True
```

FILE: strategy/payment/payment_strategy.py (9 lines)

```
1| from abc import ABC, abstractmethod
2|
3| from ...model.ticket import Ticket
4|
5|
6| class PaymentStrategy(ABC):
7|     @abstractmethod
8|     def process_payment(self, ticket: Ticket, amount: float) -> bool:
9|         pass
```

FILE: strategy/payment/upi_payment.py (8 lines)

```
1| from ...model.ticket import Ticket
2| from .payment_strategy import PaymentStrategy
3|
4|
5| class UpiPayment(PaymentStrategy):
6|     def process_payment(self, ticket: Ticket, amount: float) -> bool:
7|         print(f"Paid Rs.{amount} for ticket {ticket.ticket_id} via UPI.")
8|         return True
```

FILE: strategy/pricing/event_based_pricing.py (24 lines)

(cont.) strategy/pricing/event_based_pricing.py

```

1| import math
2| from datetime import datetime
3|
4| from ...enums.vehicle_type import VehicleType
5| from .pricing_strategy import PricingStrategy
6|
7|
8| class EventBasedPricing(PricingStrategy):
9|     EVENT_HOURLY_RATES = {
10|         VehicleType.CAR: 50.0,
11|         VehicleType.BIKE: 30.0,
12|         VehicleType.TRUCK: 70.0,
13|     }
14|
15|     def calculate_fee(
16|         self,
17|         vehicle_type: VehicleType,
18|         entry_time: datetime,
19|         exit_time: datetime,
20|     ) -> float:
21|         duration_minutes = int((exit_time - entry_time).total_seconds() // 60)
22|         hours = math.ceil(duration_minutes / 60.0)
23|         rate_per_hour = self.EVENT_HOURLY_RATES.get(vehicle_type, 0.0)
24|         return rate_per_hour * hours

```

FILE: strategy/pricing/pricing_strategy.py (15 lines)

```

1| from abc import ABC, abstractmethod
2| from datetime import datetime
3|
4| from ...enums.vehicle_type import VehicleType
5|
6|
7| class PricingStrategy(ABC):
8|     @abstractmethod
9|     def calculate_fee(
10|         self,
11|         vehicle_type: VehicleType,
12|         entry_time: datetime,
13|         exit_time: datetime,
14|     ) -> float:
15|         pass

```

FILE: strategy/pricing/time_based_pricing.py (51 lines)

```

1| import math
2| from datetime import datetime, time, timedelta
3|
4| from ...enums.vehicle_type import VehicleType
5| from .pricing_strategy import PricingStrategy
6|
7|
8| class TimeBasedPricing(PricingStrategy):
9|     PEAK_START = time(8, 0)
10|     PEAK_END = time(17, 0)
11|
12|     def _is_peak(self, t: time) -> bool:
13|         return self.PEAK_START <= t <= self.PEAK_END
14|
15|     def calculate_fee(
16|         self,
17|         vehicle_type: VehicleType,
18|         entry_time: datetime,
19|         exit_time: datetime,
20|     ) -> float:
21|         if exit_time < entry_time:
22|             raise ValueError("Exit time before entry time")
23|
24|         duration_minutes = int((exit_time - entry_time).total_seconds() // 60)
25|         total_hours = math.ceil(duration_minutes / 60.0)
26|
27|         peak_hours = 0
28|         non_peak_hours = 0
29|
30|         cursor = entry_time.replace(minute=0, second=0, microsecond=0)
31|         for _ in range(int(total_hours)):
32|             hour_start = cursor.time()
33|             if self._is_peak(hour_start):
34|                 peak_hours += 1
35|             else:
36|                 non_peak_hours += 1
37|             cursor += timedelta(hours=1)
38|
39|         peak_rate = {
40|             VehicleType.CAR: 30.0,
41|             VehicleType.BIKE: 15.0,
42|             VehicleType.TRUCK: 50.0,
43|         }[vehicle_type]
44|
45|         non_peak_rate = {
46|             VehicleType.CAR: 20.0,
47|             VehicleType.BIKE: 10.0,
48|             VehicleType.TRUCK: 30.0,
49|         }[vehicle_type]
50|
51|         return peak_hours * peak_rate + non_peak_hours * non_peak_rate

```


FILE: factory/payment_strategy_factory.py (17 lines)

```

1| from ..enums.payment_mode import PaymentMode
2| from ..strategy.payment.card_payment import CardPayment
3| from ..strategy.payment.cash_payment import CashPayment
4| from ..strategy.payment.payment_strategy import PaymentStrategy
5| from ..strategy.payment.upi_payment import UpiPayment
6|
7|
8| class PaymentStrategyFactory:
9|     @staticmethod
10|    def get(mode: PaymentMode) -> PaymentStrategy:
11|        if mode == PaymentMode.CASH:
12|            return CashPayment()
13|        if mode == PaymentMode.UPI:
14|            return UpiPayment()
15|        if mode == PaymentMode.CARD:
16|            return CardPayment()
17|        raise ValueError(f"Unsupported payment mode: {mode}")

```

FILE: factory/pricing_strategy_factory.py (14 lines)

```

1| from ..enums.pricing_strategy_type import PricingStrategyType
2| from ..strategy.pricing.event_based_pricing import EventBasedPricing
3| from ..strategy.pricing.pricing_strategy import PricingStrategy
4| from ..strategy.pricing.time_based_pricing import TimeBasedPricing
5|
6|
7| class PricingStrategyFactory:
8|     @staticmethod
9|    def get(strategy_type: PricingStrategyType) -> PricingStrategy:
10|        if strategy_type == PricingStrategyType.TIME_BASED:
11|            return TimeBasedPricing()
12|        if strategy_type == PricingStrategyType.EVENT_BASED:
13|            return EventBasedPricing()
14|        raise ValueError(f"Unsupported pricing strategy type: {strategy_type}")

```

FILE: factory/vehicle_factory.py (17 lines)

```

1| from ..enums.vehicle_type import VehicleType
2| from ..model.bike import Bike
3| from ..model.car import Car
4| from ..model.truck import Truck
5| from ..model.vehicle import Vehicle
6|
7|
8| class VehicleFactory:
9|     @staticmethod
10|    def create(number: str, vehicle_type: VehicleType) -> Vehicle:
11|        if vehicle_type == VehicleType.CAR:
12|            return Car(number)
13|        if vehicle_type == VehicleType.BIKE:
14|            return Bike(number)
15|        if vehicle_type == VehicleType.TRUCK:
16|            return Truck(number)
17|        raise ValueError(f"Unsupported vehicle type: {vehicle_type}")

```

FILE: enums/gate_type.py (6 lines)

```

1| from enum import Enum
2|
3|
4| class GateType(Enum):
5|     ENTRY = "ENTRY"
6|     EXIT = "EXIT"

```

FILE: enums/payment_mode.py (7 lines)

```

1| from enum import Enum
2|
3|
4| class PaymentMode(Enum):
5|     CASH = "CASH"
6|     UPI = "UPI"
7|     CARD = "CARD"

```

FILE: enums/payment_status.py (7 lines)

```

1| from enum import Enum
2|
3|
4| class PaymentStatus(Enum):
5|     PENDING = "PENDING"
6|     SUCCESS = "SUCCESS"
7|     FAILED = "FAILED"

```

FILE: enums/pricing_strategy_type.py (6 lines)

```

1| from enum import Enum
2|
3|
4| class PricingStrategyType(Enum):
5|     TIME_BASED = "TIME_BASED"
6|     EVENT_BASED = "EVENT_BASED"

```

FILE: enums/vehicle_type.py (7 lines)

```

1| from enum import Enum
2|
3|
4| class VehicleType(Enum):
5|     CAR = "CAR"
6|     BIKE = "BIKE"
7|     TRUCK = "TRUCK"

```

FILE: utils/date_time_parser.py (7 lines)

```
1| from datetime import datetime
2|
3|
4| class DateTimeParser:
5|     @staticmethod
6|     def parse(input_str: str) -> datetime:
7|         return datetime.strptime(input_str, "%d %b %l:%M %p %Y")
```